

■ RRC SUMMER SCHOOL · PART II

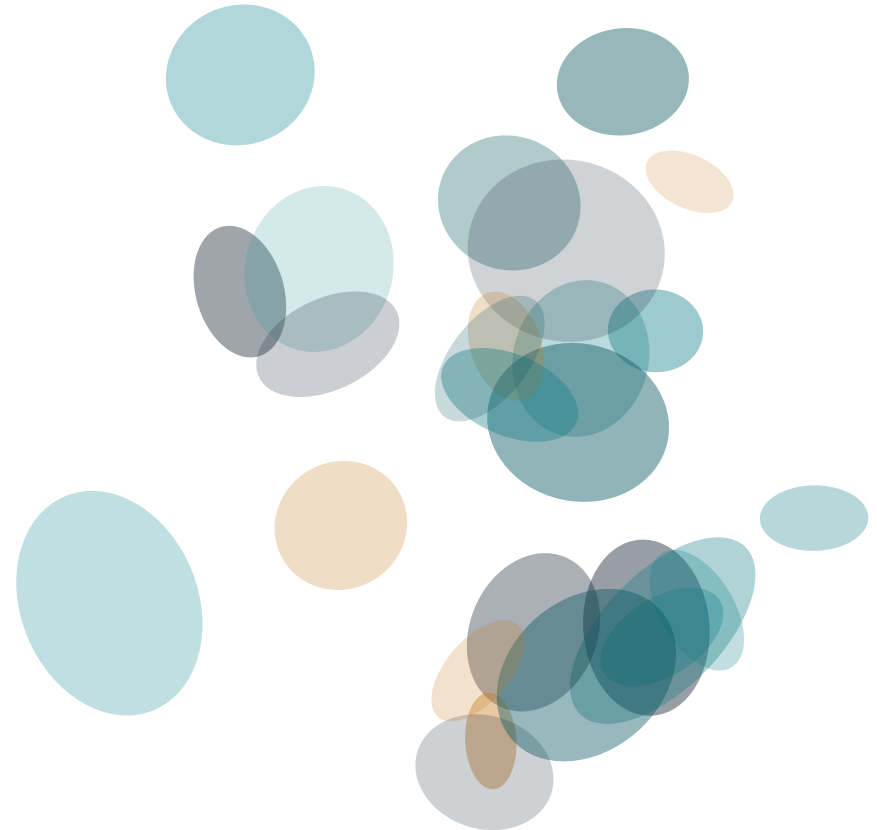
Neural Scene Models

NeRF, Gaussian Splatting, and 3D Vision for Robotics

Parts 6–9: from implicit geometry to photorealistic, deployable scene representations — and which one your robot should use.

Akash Kumbar

Robotics Research Center



From geometry to appearance to deployment



- **So far:** we stored geometry (points, voxels, meshes) and learned functions that answer geometry queries (occupancy, SDF).
- **The missing piece:** appearance. Implicit fields give shape, but not photorealistic, view-dependent color.
- **Parts 6–7:** NeRF and 3D Gaussian Splatting reconstruct how a scene looks from posed images.
- **Parts 8–9:** how we measure all of this, and which representation a real robot should actually deploy.

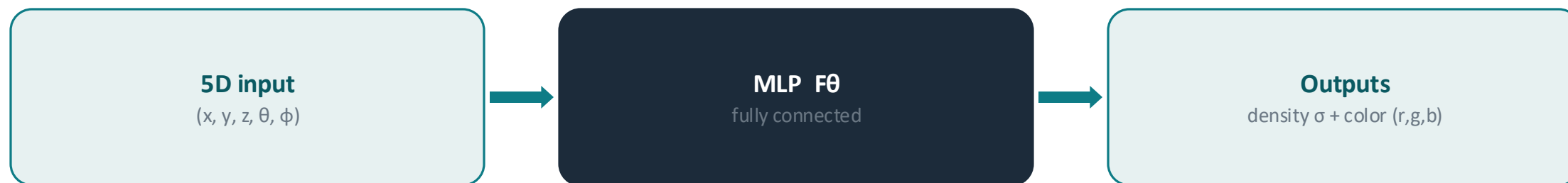


06

NeRF — Neural Radiance Fields

A tiny network that turns posed photos into a photorealistic, view-dependent 3D scene.

What is a NeRF?

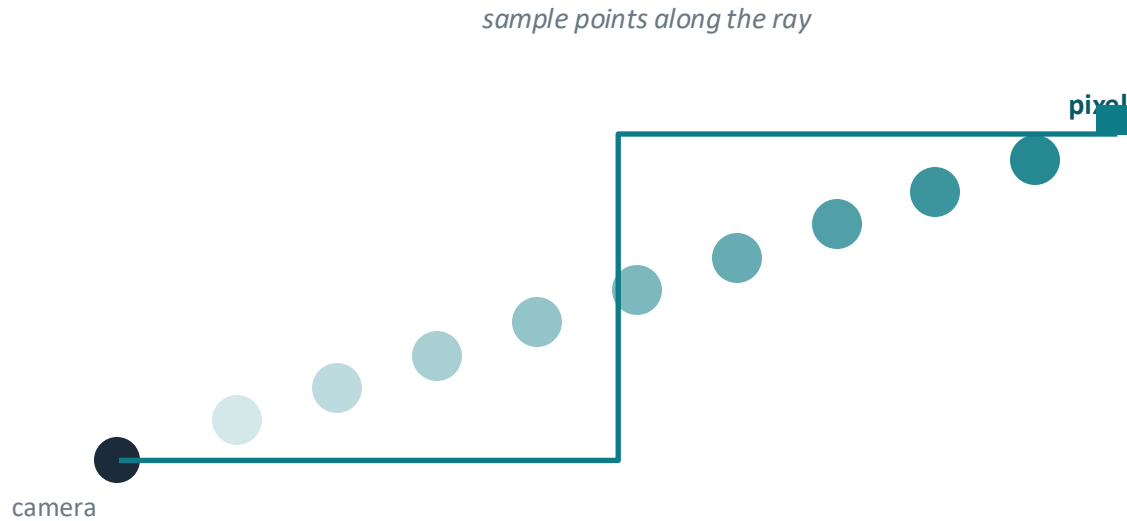


- **Input:** a 3D position (x, y, z) plus a 2D viewing direction (θ, ϕ) .
- **Output:** volume density σ (is there stuff here?) and view-dependent RGB color.
- **The whole scene** is stored implicitly in the weights of one small network.
- **Trained** to reproduce a set of input photos — no mesh, no point cloud.

Takeaway A NeRF is a function from “where am I looking” to “what color and how dense” — the scene IS the network.

Mildenhall et al., NeRF, ECCV 2020

Volume rendering: turning a field into a pixel

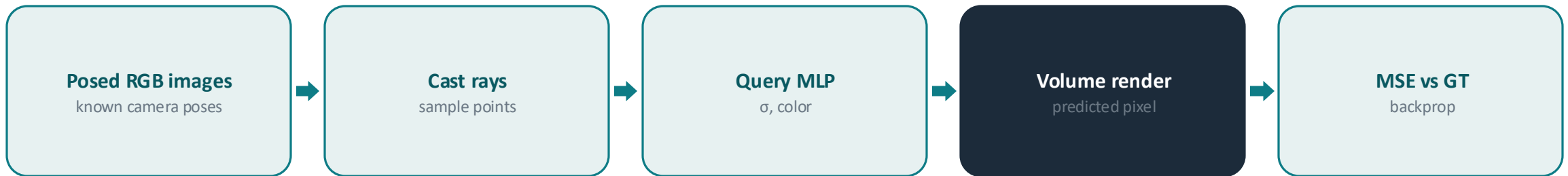


$$C(r) = \int T(t) \cdot \sigma(t) \cdot c(t) dt$$

- **March a ray** from the camera through each pixel.
- **At each sample** query the MLP for density σ and color c .
- **Composite front-to-back:** $T(t)$ is how much light survives to t .
- **The integral** gives the final pixel color — fully differentiable.

Dense, opaque samples contribute most; the rest is “see-through”.

How a NeRF trains: only posed images



- **Supervision:** compare each rendered pixel to the true pixel with mean-squared error.
- **No 3D labels needed:** geometry emerges purely from matching 2D photos across views.
- **Requires camera poses,** usually recovered with COLMAP (structure-from-motion).
- **One model = one scene:** the network overfits a single scene by design.

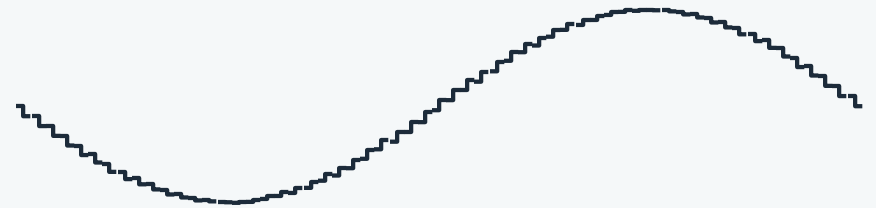
Takeaway *NeRF is self-supervised by photographs: render, compare to the photo, and backpropagate the difference.*

Positional encoding: recovering fine detail

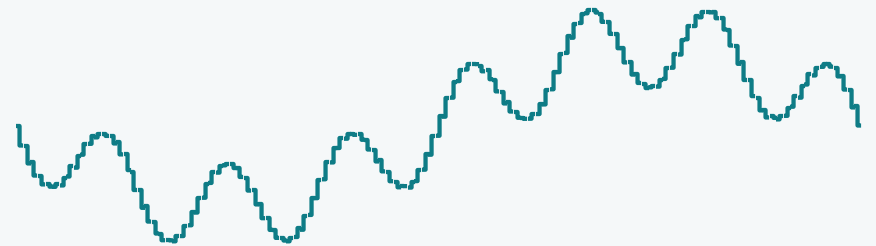
- **Problem:** MLPs are biased toward smooth, low-frequency functions.
- **Raw (x,y,z) inputs** produce blurry reconstructions — edges and texture are lost.
- **Fix:** map coordinates through high-frequency sinusoids before the MLP.
- **Effect:** the network can now represent sharp, high-frequency detail.

$$\gamma(x) = [\sin(2^k \pi x), \cos(2^k \pi x)]_k$$

Without encoding (blurry)



With Fourier features (sharp)



NeRF: strengths and weaknesses

Strengths

- Photorealistic novel-view synthesis
- Compact — scene fits in a small MLP
- RGB-only: no depth sensor needed
- Handles transparent / specular surfaces

Weaknesses

- Slow to train and to render (vanilla)
- One model per scene
- Static scenes — dynamics break it
- No clean explicit geometry / mesh

Takeaway *NeRF nails appearance, but speed, dynamics, and explicit geometry are open problems — which the variants attack.*

The NeRF variants landscape

Instant-NGP

Hash encoding → trains in seconds, renders in ms

Müller et al., 2022

Mip-NeRF 360

Handles unbounded, 360° outdoor scenes

Barron et al., 2022

Nerfacto

Nerfstudio's practical, robust default model

Tancik et al., 2023

D-NeRF / Dynamic

Adds time to model moving, deforming scenes

Pumarola et al., 2021

Takeaway *The field is racing on three fronts: speed (Instant-NGP), scale (Mip-NeRF 360), and dynamics (D-NeRF).*



07

3D Gaussian Splatting

Drop the ray-marching MLP: represent the scene as millions of explicit, rasterized 3D Gaussians.

The core idea: primitives, not rays

NeRF

Query a network along every ray and march through a continuous field. Accurate, but slow.

3D Gaussian Splatting

Fill space with explicit 3D Gaussians and rasterize them directly. Real-time and editable.



Each Gaussian is a soft, colored ellipsoid; together, millions of them reconstruct the scene.

Takeaway *Same goal as NeRF — reproduce posed images — but an explicit primitive you can sort and rasterize fast.*

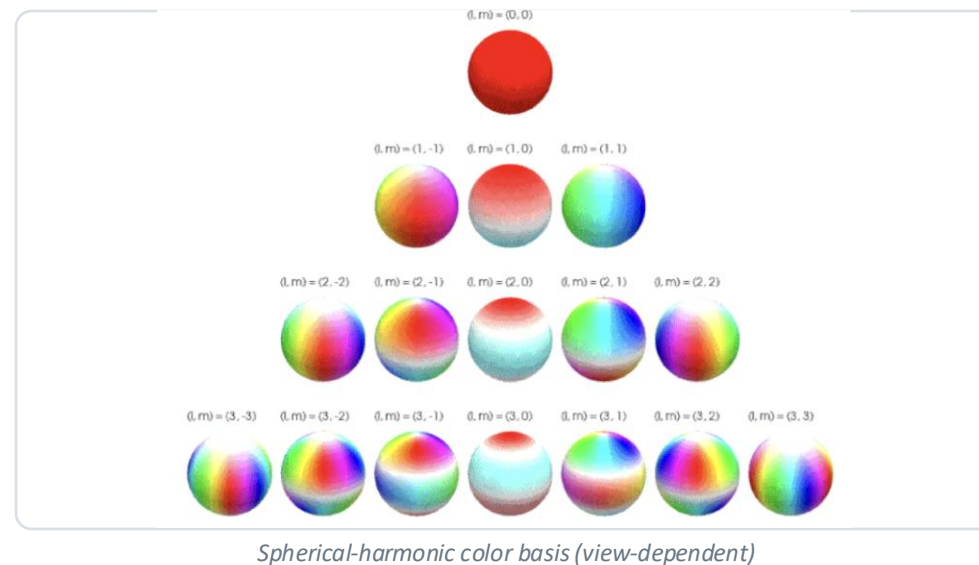
What is in one Gaussian?

Mean μ 3D position of the splat

Covariance $\Sigma = R S S^T R^T$ shape & orientation (rotation R, scale S)

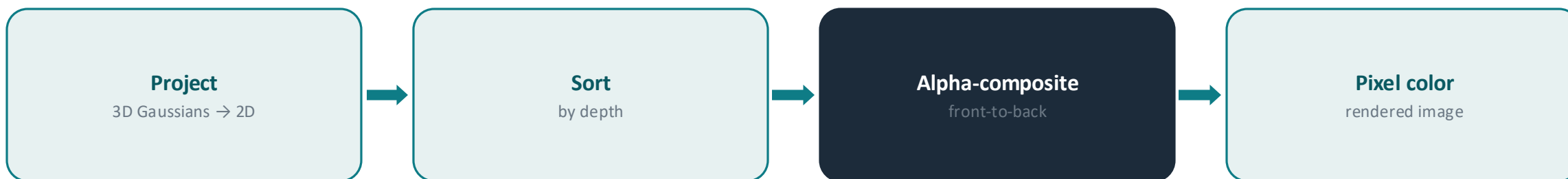
Opacity α how transparent the splat is

SH coefficients view-dependent color (spherical harmonics)



$$\frac{1}{(2\pi)^{3/2} |\Sigma|^{1/2}} \exp \left(-\frac{1}{2} (\mathbf{x} - \mu)^T \Sigma^{-1} (\mathbf{x} - \mu) \right)$$

Differentiable rasterization (why it is fast)

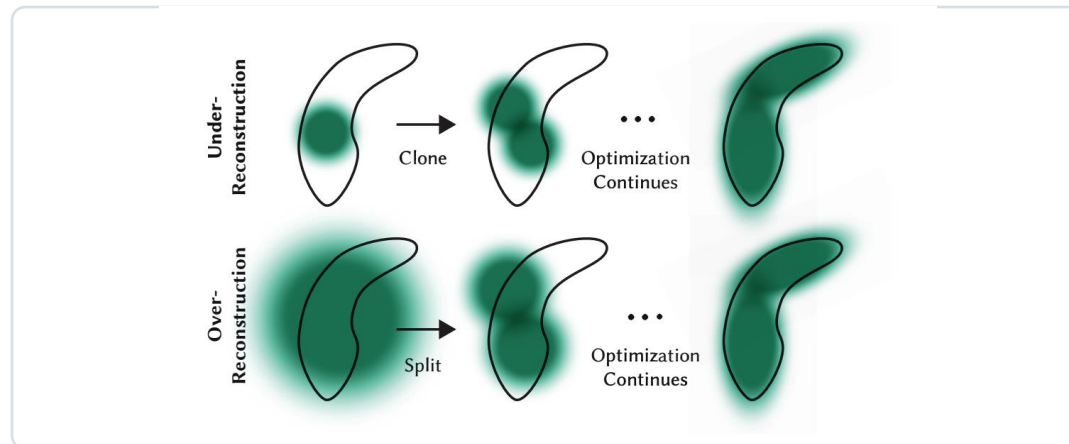


- **Project** every 3D Gaussian onto the image plane as a 2D ellipse.
- **Sort** all splats by depth — cheap, done per tile.
- **Blend** front-to-back with their opacities to get each pixel.
- **This is rasterization, not ray marching** — which is why it renders in real time.
- **Fully differentiable:** gradients flow back to every Gaussian parameter.

Takeaway *Rasterizing sorted primitives is GPU-native and parallel — the key to real-time rendering.*

Training: initialization and densification

- **Initialize** from a sparse SfM (COLMAP) point cloud.
- **Optimize** position, covariance, opacity, and SH color by comparing renders to photos.
- **Densify**: clone Gaussians in under-reconstructed areas, split oversized ones.
- **Prune**: delete near-transparent Gaussians to stay efficient.



Adaptive density control: clone, split, prune

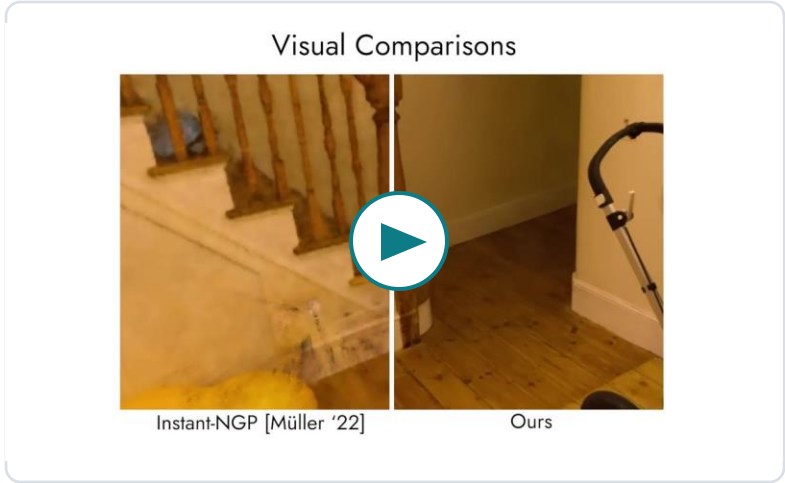


Optimization time-lapse

$$L = (1 - \lambda) L_1 + \lambda L_{D-SSIM}$$

NeRF vs 3D Gaussian Splatting

	NeRF	3D Gaussian Splatting
Representation	implicit MLP field	explicit 3D Gaussians
Rendering	ray marching	tile rasterization
Training speed	slow (fast with hash grids)	fast (minutes)
Render speed	slow → interactive	real-time (100s of FPS)
Editability	hard (baked in weights)	easy (move/add/remove splats)
Memory	compact	grows with # Gaussians



Instant-NGP vs 3DGS render

Takeaway *3DGS wins on speed and editability; NeRF variants can be more compact and geometry-faithful.*



08

Losses & Evaluation

How we train these models and how we judge whether the geometry and the pixels are actually any good.

Losses for point clouds and voxels

Cross-entropy / Focal loss

$$CE = -\sum y_i \log p_i$$

Classification & segmentation. Focal loss down-weights easy points to fight class imbalance.

Chamfer Distance (CD)

$$\sum \min_j \|x_i - y_j\|^2 + \sum \min_i \|x_i - y_j\|^2$$

Nearest-neighbor distance both ways between two point sets. Fast, the default for shape reconstruction.

Earth Mover Distance (EMD)

$$\min_{\varphi} \sum \|x_i - y_{\varphi(i)}\|$$

Optimal one-to-one assignment between sets. Stricter and more faithful than CD, but costlier.

When to use which

CE/Focal for labels; Chamfer for quick reconstruction; EMD when distribution matching matters.

Losses for implicit fields

Occupancy: BCE

$$\text{BCE}(f_{\theta}(x), y)$$

Binary cross-entropy between predicted occupancy probability and the true inside / outside label.

SDF: L1 / L2

$$|f_{\theta}(x) - s^*|$$

Regress the signed-distance value directly. L1 is robust; L2 penalizes large errors more.

Eikonal regularizer

$$\|\nabla f_{\theta}(x)\| = 1$$

A true SDF has unit-norm gradient everywhere. This keeps the field a valid distance function — smooth normals, stable surface.

Takeaway *The Eikonal term is what makes a learned field behave like a real distance function, not just any level set.*

Losses for radiance fields (NeRF & 3DGS)

Photometric loss

$$\text{L2 / L1 on pixel RGB}$$

The core signal: match each rendered pixel to the ground-truth photo. (3DGS adds a D-SSIM term.)

Perceptual / LPIPS

$$\text{LPIPS}(\hat{I}, I)$$

Compares deep features, not raw pixels — better matches what humans perceive as sharp.

Depth supervision

$$|\hat{d} - d|$$

When depth is available (RGB-D), it anchors geometry and kills floaters.

Distortion loss

$$(\text{Mip-NeRF 360})$$

Encourages density to concentrate on a clean surface — removes foggy, semi-transparent artifacts.

Evaluation metrics: geometry

Metric	Measures	Note
Chamfer Distance	mean nearest-neighbor distance between point sets	lower is better; sensitive to outliers
Hausdorff Distance	the single worst-case point error	lower is better; a strict upper bound
F-score @ τ	precision/recall of points within threshold τ	higher is better; intuitive, τ -dependent
Volumetric IoU	overlap of predicted vs true occupied volume	higher is better; for watertight shapes

Takeaway *Chamfer for average quality, Hausdorff for worst case, F-score for an interpretable percentage, IoU for volume.*

Evaluation metrics: rendering quality

PSNR

Peak Signal-to-Noise Ratio

Pixel-level error (from MSE). Higher is better.
Simple, but can reward blurry results.

higher ↑

SSIM

Structural Similarity

Compares luminance, contrast, and structure locally. Range -1 to 1; higher is better.

higher ↑

LPIPS

Learned Perceptual Similarity

Distance in a deep network's feature space.
Best matches human perception. Lower is better.

lower ↓

Takeaway Report all three: PSNR can be misleading for blurry reconstructions — LPIPS catches what the eye sees.

Zhang et al., LPIPS, CVPR 2018



09

3D Vision for Robotics

The payoff: which representation should an actual robot deploy — and for which job?

The representation landscape — a cheat sheet

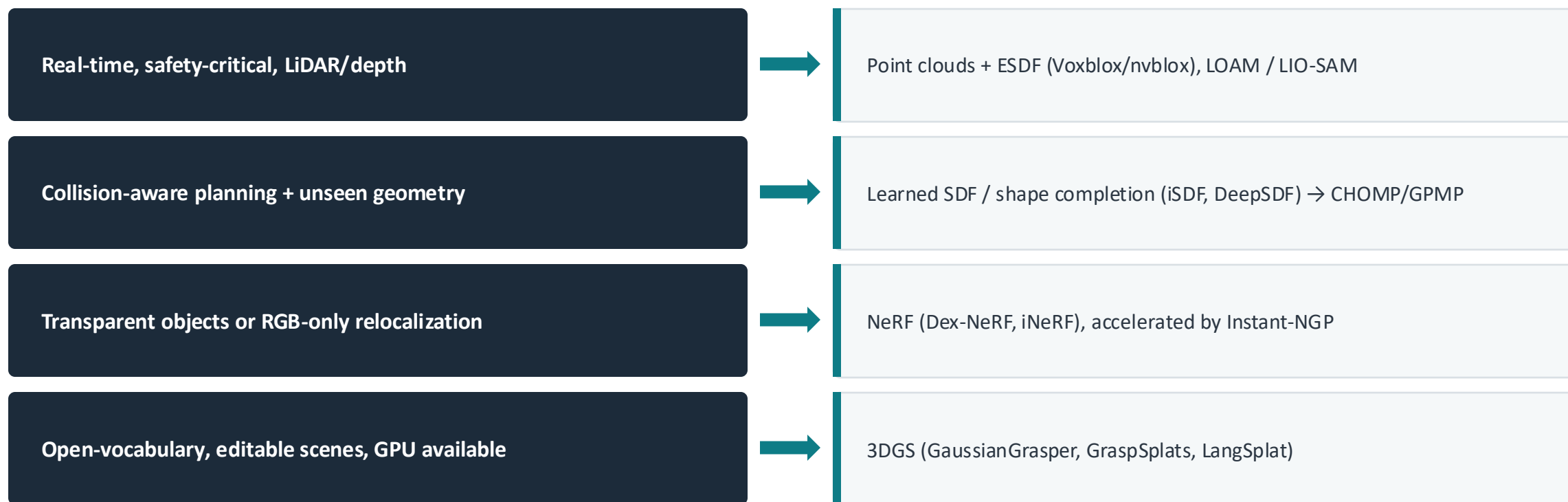
Representation	Memory	Resolution	Geometry	Render	Typical robotics use
Point cloud	grows w/ points	sensor-limited	NN distances	—	SLAM, detection, grasping
Voxel / TSDF	cubic R^3	fixed grid	good (SDF)	—	occupancy maps, navigation
Mesh	moderate	explicit	clean surface	fast	simulation, collision
Occupancy / SDF	compact MLP	continuous	best gradients	—	collision-aware planning
NeRF	compact MLP	continuous	implicit density	slow→fast	RGB-only relocalization
3D Gaussian Splatting	grows w/ splats	continuous	noisier	real-time	dense SLAM, editable maps

Which representation for which robot task?

Robot task	Representation	Why / example systems
Navigation & obstacle maps	voxel / ESDF	fast distance queries — Voxblox, nvblox, OctoMap
LiDAR SLAM	point cloud	sensor-native odometry — LOAM, LIO-SAM
Grasping (known/partial)	point cloud / SDF	surface & completion — GraspNet, DeepSDF, iSDF
Motion planning	SDF (distance+gradient)	what CHOMP / GPMP consume directly
Transparent / specular objects	NeRF	view-dependent density — Dex-NeRF, GraspNeRF
Scene-level reconstruction	3DGS / NeRF	photorealistic dense maps — SplatAM, MonoGS
Language-commanded manipulation	3DGS feature field	open-vocab grasping — LangSplat, GaussianGrasper

Representation follows the query

Don't start from the representation — start from what your downstream module needs to consume.



Decision rule synthesized from current robotics-3D literature (see references)

Open challenges in robotics 3D vision

Dynamic scenes

Most NeRF/SDF methods assume the world is static; moving objects force re-training.

Real-time on embedded HW

“Real-time rendering” (400 FPS) $\neq$ real-time tracking on robot-grade compute.

Sim-to-real gap

3D models trained in simulation transfer imperfectly to noisy real sensors.

Long-horizon spatial memory

Persisting and updating large maps over long missions.

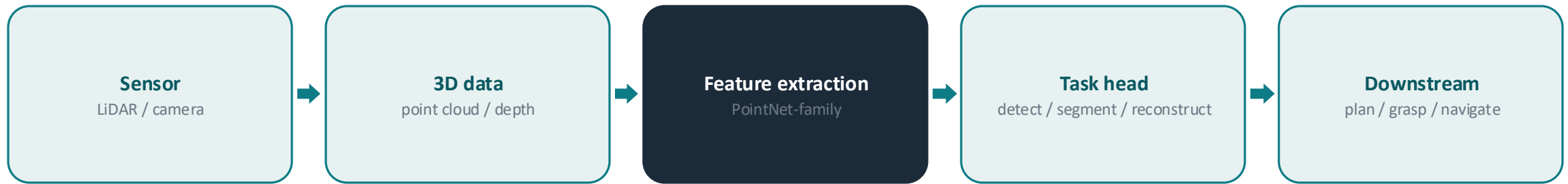
Language-grounded 3D

Connecting 3D scenes to open-vocabulary commands — the LLM + 3D frontier.

The 2025–26 trend

Convergence — hybrid maps that fuse explicit geometry for control with neural/semantic fields for understanding.

Putting it together: a robot perception stack



Where each representation lives:

- **Explicit (point cloud / voxel / SDF):** the real-time perception, mapping, and control loop.
- **Implicit (occupancy / SDF):** collision queries for the planner.
- **Neural scene models (NeRF / 3DGS):** reconstruction, relocalization, and semantic understanding.
- **The practical answer is hybrid:** explicit geometry for control, neural fields for understanding.

Five takeaways

1

Representation is a design decision

Pick it from the downstream query, not from aesthetics.

2

Explicit formats dominate robotics today

Point clouds and SDF/occupancy run the real-time control loop.

3

PointNet showed sets can be learned end-to-end

Permutation-invariant networks on raw points.

4

Implicit fields decouple resolution from memory

Geometry as a queryable function, not stored cells.

5

NeRF & 3DGS close the loop to photorealism

Neural scene models bring appearance — and new robot capabilities.

References — neural scenes & metrics

Mildenhall et al.. *NeRF: Neural Radiance Fields*. **ECCV 2020**

Müller et al.. *Instant-NGP: Multiresolution Hash Encoding*. **SIGGRAPH 2022**

Barron et al.. *Mip-NeRF 360: Unbounded Anti-Aliased NeRF*. **CVPR 2022**

Tancik et al.. *Nerfstudio / Nerfacto*. **SIGGRAPH 2023**

Pumarola et al.. *D-NeRF: Neural Radiance Fields for Dynamic Scenes*. **CVPR 2021**

Kerbl et al.. *3D Gaussian Splatting for Real-Time Radiance Fields*. **SIGGRAPH 2023**

Park et al.. *DeepSDF*. **CVPR 2019**

Mescheder et al.. *Occupancy Networks*. **CVPR 2019**

Zhang et al.. *LPIPS: The Unreasonable Effectiveness of Deep Features*. **CVPR 2018**

Fan et al.. *A Point Set Generation Network (Chamfer / EMD)*. **CVPR 2017**

References — robotics systems

Fang et al.. *GraspNet-1Billion*. **CVPR 2020**

Zhang & Singh. *LOAM: LiDAR Odometry and Mapping*. **RSS 2014**

Shan et al.. *LIO-SAM*. **IROS 2020**

Oleynikova et al.. *Voxblox (incremental ESDF)*. **IROS 2017**

Ortiz et al.. *iSDF: Real-Time Neural Signed Distance Fields*. **RSS 2022**

Yen-Chen et al.. *iNeRF: Inverting Neural Radiance Fields for Pose*. **IROS 2021**

Ichnowski et al.. *Dex-NeRF: Grasping Transparent Objects*. **CoRL 2021**

Keetha et al.. *SplaTAM: Splat, Track & Map*. **CVPR 2024**

Matsuki et al.. *Gaussian Splatting SLAM (MonoGS)*. **CVPR 2024**

Zheng et al.. *GaussianGrasper*. **RA-L 2024**

Qin et al.. *LangSplat: 3D Language Gaussian Splatting*. **CVPR 2024**

Kerr et al.. *LERF: Language Embedded Radiance Fields*. **ICCV 2023**

Figures and clips adapted from the RRC Gaussian-Splatting deck and the original papers, for educational use.